BITÁCORA DE PROYECTO

Rivales FC

Informe de evolución · MK1 ⟶ MK5

Núcleo local · Online 1v1 por UDP · libGDX + Box2D

Evolución de un vistazo

Versión	Objetivo	Principales incorporaciones
MK1	Prototipo	Física, movimiento, disparos, arquero y split-screen
MK2	Jugabilidad y estabilidad	Rediseño gráfico, corrección de física, barrida, lobby e IA
MK3	Experiencia y feedback	Audio, screen shake, animaciones, posesión y tutorial
MK4	Cierre de pendientes y pulido	Correcciones de entrada y potencia, estirada del arquero, IA con decisiones, repetición de gol, estadísticas y MVP, shader de blur, opciones persistentes
MK5	Online 1v1 y rebalanceo	Online 1v1 por UDP (host + cliente), interpolación, ping y revancha, protocolo v2, 1 jugador a pantalla completa, barrida nerfeada y tiro buffeado

Cada versión responde a lo que dejó la anterior: la **MK1** fijó la base física y de control, la **MK2** resolvió los problemas técnicos y sumó mecánicas, la **MK3** mejoró la respuesta audiovisual y, finalmente, la **MK4** cerró los pendientes, corrigió errores que afectaban la sensación de juego y dejó el núcleo local listo para pasar a red. La MK5 dio ese paso: sumó el modo online 1v1 por red, lo pulió con interpolación, ping y revancha, y rebalanceó la barrida y el tiro.

MK1 Versión inicial / Prototipo

Funcionalidades implementadas

- Movimiento básico de los jugadores y sistema de disparo utilizando **Box2D**.
- Implementación de pantalla dividida (**split-screen**), con una cámara independiente para cada jugador.
- Primer prototipo de comportamiento del arquero, basado en la posición de la pelota.

Problemas detectados

Durante esta etapa se identificaron varios problemas que limitaron la estabilidad y la experiencia de juego:

- **Flujo de juego incompleto:** el partido comenzaba directamente, sin menú, selección de modalidad ni opciones de reinicio.
- **Problemas al producirse un gol:** la simulación podía bloquearse o finalizar de forma inesperada.
- **Problemas de renderizado:** los jugadores aparecían invertidos y los sprites tenían poco nivel de detalle.
- **Comportamiento del arquero:** la reacción inmediata ante la posición de la pelota hacía que prácticamente todos los remates fueran bloqueados.
- **Aspectos visuales:** el césped, la pelota y el público presentaban un nivel de detalle reducido y algunos errores de representación.

MK2 Mejora de jugabilidad, estabilidad y presentación

1. Rediseño gráfico

Se corrigió el problema de inversión de los sprites, provocado por la diferencia entre el sistema de coordenadas de `Pixmap` y el del mundo de Box2D. La solución se implementó en `PixelArtFactory`, invirtiendo correctamente las coordenadas verticales durante el renderizado. También se mejoraron los recursos gráficos:

- Los sprites de los jugadores pasaron de **16×28 px** a **20×32 px**.
- Se incorporó sombreado para generar mayor sensación de volumen.
- Se agregaron ojos y contornos oscuros para separar a los personajes del césped.
- Se creó un diseño específico para el arquero, con camiseta negra y guantes amarillos.
- Se mejoró la textura de las tribunas mediante `buildCrowdTexture`.
- Se aumentó el tamaño y contraste de la pelota.
- Se incorporó textura al césped para evitar una superficie completamente uniforme.

2. Corrección del problema producido por los goles

El problema estaba relacionado con la modificación de cuerpos físicos desde `beginContact()` mientras Box2D ejecutaba `world.step()`. Para solucionarlo se implementó un **sistema de acciones diferidas**:

- `beginContact()` únicamente establece la bandera `goalPending`.
- Las modificaciones de la simulación se realizan después, desde `fixedTick()`.
- Una vez finalizado `world.step()`, se reposicionan los jugadores y la pelota y se actualiza el estado del partido.

Esto evitó modificar el mundo físico durante la ejecución interna de Box2D y mejoró considerablemente la estabilidad de la simulación.

3. Ajustes del arquero

Se modificó el comportamiento del arquero para que no reaccionara de manera instantánea a la pelota:

- Incorporación de un tiempo de reacción mediante `GK_REACTION_TIME`.
- Reducción de la velocidad máxima de **6,5 m/s a 4,2 m/s**.
- Aumento del margen de tolerancia usado para determinar si la pelota está cubierta.

Con estos cambios, los remates dirigidos hacia los extremos del arco tienen mayores posibilidades de superar al arquero.

4. Nueva mecánica: barrida

Se incorporó la mecánica de barrida (*slide tackle*), gestionada mediante `slidePressed` como evento de activación de un solo cuadro.

Elemento	Detalle
Controles	Jugador 1: **Ctrl izquierdo** · Jugador 2: **Shift derecho**
Dirección	Se utiliza la última dirección de movimiento registrada; no se puede modificar durante la acción
Velocidad y duración	9,5 m/s durante 0,35 s (`SLIDE_DURATION`)
Recuperación	1 s de espera tras finalizar (`SLIDE_COOLDOWN`)
Contacto con la pelota	`canWinBallThisSlide()` decide si se disputa la posesión y `MatchSimulation.handleSlideTackle()` aplica el impulso
Feedback visual	Rotación del sprite y estela de polvo (`MotionTrail`)

5. Menú e inteligencia artificial

Se incorporó una pantalla inicial (`LobbyScreen`) para elegir entre `ONE_PLAYER` y `TWO_PLAYERS`, desarrollada con `ShapeRenderer` y `BitmapFont` para mantener la estética del juego.

Para el modo de un jugador se implementó `SupportAI`, que controla al segundo jugador generando entradas mediante `PlayerInput`, el mismo sistema que usan los controles humanos. Así, movimiento, disparo y barrida mantienen **una única implementación** sin importar si la entrada viene del teclado o de la IA.

MK3 Audio, feedback y mejoras de experiencia

La MK3 se centró en mejorar la respuesta del juego ante las acciones del jugador y en aportar más información durante y después del partido.

1. Sistema de audio

Se incorporó `AudioFactory.java`, que genera los sonidos en tiempo de ejecución mediante ondas cuadradas y triangulares, exportadas como WAV, sin archivos de audio externos.

Evento	Sonido
Pateo	Grave para pases suaves y agudo para remates fuertes
Barrida	Impacto cuando la acción consigue recuperar la pelota
Silbato	Al comenzar una jugada, después de un gol y al inicio del segundo tiempo
Gol	Fanfarria
Final del partido	Bocinazo

2. Screen shake

Efecto de movimiento de cámara que refuerza los impactos importantes, aplicado a las cámaras de los paneles del split-screen. Se activa en remates con potencia superior a 0,75, barridas que ganan la pelota y goles (con mayor intensidad).

3. Animación de movimiento

Movimiento vertical (*bobbing*) con una deformación leve (*squash*), cuya intensidad y frecuencia dependen de la velocidad del personaje. Evita el desplazamiento rígido y mejora la sensación de movimiento sin animación cuadro por cuadro.

4. Indicador de posesión

`MatchSimulation` comenzó a registrar qué equipo mantiene la pelota cerca en cada ciclo de simulación. A partir de eso se incorporó una **barra de posesión azul y roja** bajo el marcador y el **porcentaje de posesión** en la pantalla final.

5. Tutorial durante el partido

Durante el saque inicial, mientras el marcador está 0-0 y la jugada no comenzó, se muestran los controles de cada jugador, para conocer las acciones principales desde el propio juego.

MK4 Cierre de pendientes, corrección de errores y camino a la versión final

La MK4 tuvo tres frentes: **corregir errores heredados de la MK3** que afectaban la sensación de juego sin que se notara la causa, **resolver los cuatro pendientes** que la MK3 había dejado fuera de alcance y **sumar funcionalidades nuevas** de cara a la versión final.

16 → 20	2.808 → 4.011	5	4
archivos Java	líneas de código (+43 %)	errores corregidos	pendientes de MK3 resueltos

1. Diferencias entre MK3 y MK4

Comparación directa entre los dos proyectos entregados:

Área	MK3	MK4
Potencia de pateo	Siempre 0: todos los pateos salían como pase mínimo	Potencia real de 0 a 1, consumida al patear
Pateos y barridas	Cerca de la mitad de los eventos se perdía	Eventos “pegajosos” hasta ser consumidos por la simulación
Impulsos de pelota	3 – 16: el tope de Box2D se alcanzaba con poca carga	1,1 – 2,9 (≈ 14 a ≈ 38 m/s): la carga importa

Área	MK3	MK4
Arquero	Un solo estado: seguir la Y de la pelota	TRACK / DIVE con predicción, reflejo, recarga y animación
IA de soporte	Correr a la pelota y patear (98 líneas)	Máquina DEFEND / CHASE / CARRY con remate al rincón (227 líneas)
Fin del partido	Resultado y porcentaje de posesión	Panel con 6 estadísticas, notas 1–10 y mejor jugador
Desenfoque del rival	Capas translúcidas superpuestas	Shader GLSL gaussiano 5×5, con las capas como respaldo
Gol	Cartel genérico y festejo de 2,5 s	Goleador nombrado, confeti, ovación y repetición en cámara lenta
Opciones	Ninguna: dificultad única y tiempo fijo de 3 min	Dificultad, duración y volumen, guardados entre sesiones
Pausa y sonido	Cartel “PAUSA”	Menú de pausa, tecla **M** para silenciar y 3 sonidos nuevos
Compilación	Sin codificación forzada	UTF-8 forzado en Gradle

2. Errores heredados de la MK3 y cómo se corrigieron

Ninguno de estos errores hacía fallar el juego, pero todos deterioraban el “feel”: los remates no dependían de la carga y algunas acciones simplemente no respondían.

2.1 La potencia de pateo era siempre nula

- **Causa:** `PlayerEntity.applyInput` reiniciaba la carga a 0 en el mismo tick del “soltar”, justo antes de que `handleKick` leyera la potencia. Todos los pateos salían con el mínimo.
- **Solución:** la carga ya no se borra en el tick del soltado (`else if (!input.kickReleased)`). Se creó `consumeKickPower()`, que devuelve la potencia acumulada y reinicia la carga, y `handleKick` la llama al patear.

2.2 Se perdían pateos y barridas

- **Causa:** el juego renderiza a 60 fps pero simula a 30 Hz. `pollInput()` asignaba `kickReleased` y `slidePressed` en cada frame; si el evento caía en un frame sin tick de simulación, el frame siguiente lo pisaba con `false`. Se perdía cerca del 50 %.
- **Solución:** los eventos pasaron a ser “pegajosos”: se activan con `= true` y no se sobrescriben. Permanecen hasta que `MatchSimulation` los consume y limpia con `clearTransient()`, incluso durante la repetición.

2.3 La escala de los impulsos anulaba el efecto de la carga

- **Causa:** la pelota pesa ≈ 0,076 kg (densidad 0,5 × área del círculo de radio 0,22). Con la simulación a 30 Hz, Box2D limita el desplazamiento a 2 m por paso, es decir ≈ 60 m/s. Con impulsos de 3 a 16, el pase mínimo ya era de ≈ 39 m/s y con un 12 % de carga se llegaba al tope: cargar más no cambiaba nada.

- **Solución:** se reescalaron dos constantes en `Constants.java`: `PASS_IMPULSE_MIN` de 3 a **1,1** y `SHOT_IMPULSE_MAX` de 16 a **2,9**. El rango útil pasó a ≈ 14 – 38 m/s, y toda la barra de carga tiene efecto.

2.4 Estado residual al reposicionar tras un gol

- **Causa:** una barrida o una carga de remate en curso no se limpiaba al reposicionar para el saque.
- **Solución:** nuevo método `PlayerEntity.resetState()` (anula barrida, recuperación y carga), invocado desde el reposicionamiento de `MatchSimulation`.

2.5 Codificación de caracteres

- **Causa:** el código contiene tildes y ñ; en Windows (cp1252) el compilador podía fallar o corromper textos.
- **Solución:** `build.gradle` fuerza `options.encoding = 'UTF-8'` en todas las tareas `JavaCompile`.

3. Pendientes de la MK3 resueltos

3.1 Estirada explícita del arquero

`GoalkeeperEntity` pasó de un único estado a dos: **TRACK** (seguimiento con tiempo de reacción, como en MK2/MK3) y **DIVE** (estirada). Su firma ahora recibe posición y velocidad de la pelota, y un indicador `canDive` que lo desactiva fuera de juego.

- **Detección:** en cada tick evalúa si la pelota está delante del arco a menos de 16 m, se acerca a más de 6 m/s y el arquero no está en recarga.
- **Predicción:** calcula el tiempo de llegada (distancia ÷ velocidad hacia el arco) y proyecta la posición Y en trayectoria recta. Solo se tira si el remate va al arco y no llega caminando (diferencia mayor a 0,7 m).
- **Decisión:** aplica un reflejo de 0,10 s escalado por dificultad. Luego sortea si se tira (50 / 75 / 92 % según el nivel); si no, “se duerme” y queda 0,8 s en recarga. Si se tira, apunta a la Y prevista más un error aleatorio (±1,2 / 0,6 / 0,25 m).
- **Ejecución:** se desplaza a 9 m/s durante un máximo de 0,45 s y luego queda 1,8 s en recarga. Sigue siendo un cuerpo cinemático, con el rango vertical limitado para no salirse de la boca del arco.
- **Presentación:** `getVisualDive()` (de −1 a 1) eleva y estira el sprite hasta un 32 %, y suena un “fuuum” al iniciar la estirada.

3.2 IA de soporte con decisiones y posicionamiento

`SupportAI` pasó de “correr a la pelota y patear” (98 líneas) a una máquina de estados (227 líneas). Sigue generando un `PlayerInput`, de modo que la implementación de movimiento, pateo y barrida continúa siendo única.

Estado	Comportamiento
DEFEND	Si el rival tiene la pelota en la mitad propia y está a más de 2,4 m, se ubica entre la pelota y el arco propio. Si el rival se acerca, intenta una barrida.
CHASE	Va a buscar la pelota. A menos de 4 m se acerca desde atrás respecto de la dirección en que quiere patear, porque en este juego el pateo sale “del jugador hacia la pelota”.

Estado	Comportamiento
CARRY	Con la pelota al alcance decide entre **SHOOT** (dentro del rango de remate: 11 / 14 / 17 m según dificultad, al rincón más lejano del arquero rival, con error de puntería) y **PASS** (toque suave hacia un espacio libre, esquivando al rival si lo tiene encima).

- Antes de patear se acomoda detrás de la pelota hasta lograr alineación con el objetivo (coseno > 0,94); si tarda más de 1,4 s, replantea la jugada (anti-atasco).
- **Aclaración de alcance:** en el 2v2 fijo cada equipo tiene un solo jugador de campo, por lo que el "pase" de la IA es un toque a espacio libre y no a un compañero. Los pases reales llegarán con más jugadores por equipo.

3.3 Pantalla de mejor jugador

Se creó `MatchStats`, que acumula por equipo remates, remates al arco, recuperaciones y atajadas, y calcula las notas individuales. Cómo se registra cada dato:

- **Remate:** cada `kickReleased` con potencia ≥ 0,45 (`SHOT_MIN_POWER`) y dirección hacia el arco rival. Cuenta "al arco" si la recta proyectada cruza la línea de gol dentro del ancho del arco.
- **Recuperación:** cada barrida que gana la pelota (`handleSlideTackle`).
- **Atajada:** en `beginContact`, si la pelota toca a un arquero y hubo un remate al arco del equipo contrario en los últimos 4 s, se anota una atajada pendiente. Se contabiliza luego en `fixedTick`, reutilizando la técnica de acciones diferidas de la MK2.

Las notas van de 1 a 10 y se calculan con estas fórmulas:

Posición	Cálculo de la nota	Bono por resultado
Jugador de campo	5,5 + 1,5 por gol + 0,3 por remate al arco + 0,15 por remate + 0,5 por recuperación	+0,5 si gana · −0,3 si pierde
Arquero	5,5 + 0,7 por atajada − 0,5 por gol recibido + 1,0 si no recibe goles	+0,5 si gana · −0,3 si pierde

`drawFullTimePanel()` presenta un panel centrado con el resultado, seis estadísticas comparadas entre AZUL y ROJO (goles, posesión, remates, al arco, recuperaciones, atajadas), las notas de los cuatro jugadores y una banda dorada con el **MEJOR JUGADOR**. Se escala según la altura de la pantalla.

3.4 Desenfoque mediante shader

El desenfoque por capas translúcidas se reemplazó por un shader real, en la nueva clase `BlurShader`:

- **Técnica:** kernel gaussiano 5×5 en GLSL con promedio ponderado por alfa, para que los bordes transparentes no oscurezcan el resultado.
- **Sin FrameBuffer:** se activa con `batch.setShader(...)` solo mientras se dibuja al rival, y el blur se calcula por sprite.
- **Textura con margen** (25 %) para que el desenfoque se extienda fuera del contorno del sprite sin recortarse.
- **Radio dinámico:** 1,4 + 0,12 × velocidad (máx. 10), así el rival se ve más borroso cuanto más rápido se mueve.

- **Respaldo:** si el driver no compila el shader, `create()` devuelve `null` y la pantalla usa automáticamente el desenfoque por capas de la MK3.

4. Nuevas funcionalidades

4.1 Repetición de gol en cámara lenta

- **Grabación:** un buffer circular de 106 cuadros (`ReplayFrame`, 3,5 s a 30 Hz) guarda en cada tick de juego la posición y velocidad de pelota, jugadores y arqueros, con su estado de barrida y estirada. Solo almacena datos de presentación.
- **Captura:** al producirse el gol, `captureReplay()` copia el buffer en orden cronológico, antes de reposicionar a los jugadores.
- **Reproducción:** tras el festejo, la fase `REPLAY` no ejecuta física. Avanza un cursor a 0,6× (≈ 5,9 s en total) y aplica cada cuadro con `setTransform`.
- **Presentación:** una cámara suavizada sigue a la pelota en ambos paneles, con barras “cine”, cartel REPLAY parpadeante y barra de progreso.
- **Salida:** se puede saltear con **TAB** (`skipReplay()`). Al terminar, se reposiciona todo y se pasa al saque.

4.2 Atajadas

Contador por equipo, cartel “¡ATAJADA!” con desvanecimiento, sonido propio y un shake leve. Se dispara desde `consumeSaveEvent()`, que se consume una sola vez por evento.

4.3 Opciones y dificultad persistentes

La nueva clase `Settings` centraliza dificultad, duración de cada tiempo (1 / 2 / 3 / 5 min) y volumen (0–100 %, en pasos del 10 %). Se guardan con `Preferences` de libGDX, se cargan al iniciar (`Settings.load()`) y todos los sonidos pasan por `Settings.sfx()`. El `LobbyScreen` sumó tres filas de opciones: se navega con W/S, se cambia el valor con A/D y se confirma con ENTER (1 y 2 siguen arrancando directo).

La dificultad afecta a la IA de soporte y a los dos arqueros:

Parámetro	Fácil	Normal	Difícil
Escala de movimiento de la IA	0,80	0,92	1,00
Distancia de sprint de la IA (m)	8,0	3,5	2,5
Error de puntería de la IA (m)	1,40	0,70	0,25
Rango de remate de la IA (m)	11	14	17
Velocidad del arquero	×0,85	×1,00	×1,10
Tiempo de reacción del arquero	×1,50	×1,00	×0,70
Probabilidad de estirada	50 %	75 %	92 %
Error de estimación al tirarse (m)	1,20	0,60	0,25

4.4 Confeti, ovación y goleador nombrado

Al hacer un gol se lanzan 120 partículas de confeti con los colores del equipo goleador, dorado y blanco, más una ovación del público. El cartel ahora nombra al autor (por ejemplo, “GOL de … (AZUL)”).

4.5 Sonidos, silencio y menú de pausa

`AudioFactory` sumó tres sonidos sintetizados: la ovación (`crowdCheer`, ruido filtrado con envolvente), el “fuuum” de la estirada (`diveWhoosh`) y el toque seco de la atajada (`saveTone`). La tecla **M** silencia o reactiva el sonido y se guarda. El menú de pausa ahora lista sus acciones: continuar, reiniciar, sonido, modo debug y volver al menú.

5. Ajustes de balance

Además de lo anterior, el código muestra ajustes finos de parámetros que conviene registrar:

Parámetro	MK3 → MK4	Efecto
`SLIDE_KICK_POWER`	0,5 → 0,35	Toque más suave de la barrida sobre la pelota
`GOAL_CELEBRATION_SECONDS`	2,5 s → 2,0 s	Festejo más corto, ahora seguido de la repetición
Escala del cartel de gol	2,4 → 2,0	Texto algo más discreto

6. Mapa de cambios por archivo

Archivo	Líneas	Cambio principal
Nuevos		
`Settings.java`	0 → 104	Opciones persistentes y parámetros de dificultad
`sim/MatchStats.java`	0 → 64	Estadísticas, notas y MVP
`sim/ReplayFrame.java`	0 → 20	Cuadro de la repetición de gol
`gfx/BlurShader.java`	0 → 77	Shader GLSL de desenfoque
Modificados		
`screens/MatchScreen.java`	916 → 1.265	Panel final, repetición, confeti, cartel de atajada, blur, pausa y entrada pegajosa
`sim/MatchSimulation.java`	403 → 605	Fase REPLAY, estadísticas, atajadas y pateo con potencia consumida
`entities/GoalkeeperEntity.java`	97 → 218	Estados TRACK / DIVE
`ai/SupportAI.java`	98 → 227	Máquina DEFEND / CHASE / CARRY
`screens/LobbyScreen.java`	211 → 268	Filas de opciones y navegación
`audio/AudioFactory.java`	168 → 207	Tres sonidos nuevos
`entities/PlayerEntity.java`	172 → 195	`consumeKickPower()`, `resetState()` y fix de la carga
`Constants.java`	121 → 138	Nuevos parámetros y reescala de impulsos
`RivalesFCGame.java`	22 → 23	Carga de `Settings` al iniciar

Archivo	Líneas	Cambio principal
`build.gradle`, `README.md`	—	UTF-8 y novedades y controles actualizados

Sin cambios: `Ball`, `Field`, `GameMode`, `PlayerInput`, `MotionTrail`, `PixelArtFactory` y `DesktopLauncher`.

> **Continuidad de diseño**
> Varias soluciones de la MK4 reutilizan decisiones anteriores: las atajadas usan el mismo patrón de **acciones diferidas** de la MK2 (bandera en `beginContact()`, efecto en `fixedTick()`), y la IA sigue pasando por `PlayerInput`, por lo que no hubo que tocar el movimiento ni el pateo. La repetición graba solo datos de presentación, sin volver a simular física.

MK5 Online 1v1, interpolación en el cliente y rebalanceo de jugabilidad

La MK5 llevó Rivales FC de un núcleo solo local a un **partido en red 1 contra 1**. El equipo sumó la capa de red (Etapa 2, que en el repositorio figura como MK4.5) y, sobre esa base, la pulió con **interpolación, ping y revancha**; además **rebalanceó la barrida y el tiro** y pasó el modo de un jugador a **pantalla completa**. Este informe registra todo lo que cambió entre el código de la MK4 y el de la MK5.

20 ⟶ 26	4.011 ⟶ 5.314	6	26
archivos Java (+1 de pruebas)	líneas de código (+32 %)	clases de red nuevas	pruebas automáticas de red

1. Diferencias entre MK4 y MK5

Comparación directa entre los dos proyectos entregados:

Área	MK4	MK5
Modos de juego	1 jugador (pantalla dividida) y 2 jugadores local	Se suman **ONLINE_HOST** y **ONLINE_CLIENT** (1v1 por red); 1 jugador pasa a pantalla completa
Red	Ninguna	Host autoritativo + cliente por UDP (puerto 7777), protocolo v2 con 7 tipos de mensaje
Vista del cliente online	—	Interpolación de snapshots con 70 ms de retraso y extrapolación acotada a 100 ms
Ping	—	`PING` / `PONG` cada 1 s, suavizado 70/30, visible para host y cliente
Fin del partido online	—	Revancha con R (solo el host), sin cortar la conexión
Cámara	16 m de alto, un panel por jugador	20 m de alto con 30 % de sesgo hacia la pelota en pantalla completa; dividida solo en 2 jugadores local

Área	MK4	MK5
Rival en 1 jugador	Difuminado por shader	Nítido (`drawSharpRival`); el online mantiene el rival difuminado
Barrida	9,5 m/s · 0,35 s · recarga 1,0 s	8,0 m/s · 0,30 s · recarga 1,8 s + 0,35 s en el piso
Tiro	Impulso máx. 2,9 (≈ 38 m/s), carga 1,0 s, alcance 0,9 m, siempre en línea recta jugador → pelota	Impulso máx. 3,8 (≈ 50 m/s), carga 0,75 s, alcance 1,05 m, dirección apuntable
Lobby	5 filas (2 modos + 3 opciones)	7 filas: se suman “crear partida” y “unirse” (IP editable y recordada)
Recordatorio de controles online	Mostraba las teclas del Jugador 2 (flechas)	Muestra las correctas (WASD)
Pruebas automáticas	Ninguna	`NetSelfTest`: 26 pruebas de red por loopback

2. Online 1v1: arquitectura y protocolo

El equipo agregó la Etapa 2 del cronograma: el partido puede jugarse entre dos computadoras. La capa de red es Java puro (`java.nio`, sin LibGDX) y vive en el paquete nuevo `net`.

2.1 Host autoritativo

- **Host (equipo AZUL):** ejecuta toda la simulación (`MatchSimulation`, a 30 Hz) igual que en local y usa el input del cliente para el jugador rojo.
- **Cliente (equipo ROJO):** no simula. `MatchSimulation.step()` devuelve de inmediato en `ONLINE_CLIENT`; el estado llega por `applySnapshot()`.
- **Sin hilos:** `NetHost.poll()` y `NetClient.poll()` se llaman una vez por frame sobre un `DatagramChannel` no bloqueante.
- **Un solo rival:** el host acepta un cliente; un segundo cliente es rechazado. Si no hay rival, la simulación no avanza y la pelota queda quieta.
- **Conexión:** el cliente reenvía `HELLO` cada 0,5 s hasta recibir `WELCOME` (tiempo máximo 5 s). Cada lado da por caída la conexión tras 3 s sin recibir datos. Si el rival se cae, el host queda esperando y puede volver a entrar.

2.2 Protocolo (Protocol.java)

Todos los paquetes empiezan con una cabecera de 4 bytes: **MAGIC** `0x5246` (“RF”, 2 bytes) + **VERSION** (1) + **TIPO** (1). Los paquetes con otra marca o versión se ignoran. Puerto por defecto: **7777**; tamaño máximo de paquete: 256 bytes.

Mensaje	Sentido	Contenido
`HELLO` (1)	cliente → host	Pedido de ingreso; se reintenta hasta recibir `WELCOME`
`WELCOME` (2)	host → cliente	Ingreso aceptado; el host lo repite si el anterior se perdió
`INPUT` (3)	cliente → host	Secuencia (int) + moveX + moveY (float) + banderas (sprint, kick sostenido, kick soltado, cambio, barrida): 17 bytes en total
`BYE` (4)	ambos	Aviso de salida

Mensaje	Sentido	Contenido
`SNAPSHOT` (5)	host → cliente	Estado del partido: ≈ 138 bytes (4 de cabecera + 134 de cuerpo)
`PING` (6)	cliente → host	Marca de tiempo del cliente (int, ms) + último ping medido (short; −1 si aún no mide). Nuevo en v2
`PONG` (7)	host → cliente	Devuelve la misma marca de tiempo, sin tocarla. Nuevo en v2

2.3 Snapshot y eventos por contador

- **Contenido:** fase, mitad, marcador, posesión, tiempos, progreso de la repetición; pelota (posición, ángulo, velocidad); por jugador (posición, velocidad, barrida, carga de pateo); por arquero (Y, velocidad, estirada); y las estadísticas (remates, al arco, recuperaciones, atajadas).
- **Frecuencia:** `MatchScreen.sendSnapshotIfDue()` envía ~30 por segundo medidos por tiempo, no por tick, porque durante la repetición el tick no avanza. El campo `tick` se reemplaza por un número de secuencia propio (`netSeq`) y el cliente descarta los paquetes con número menor o igual al último.
- **Eventos por contador:** el pateo, la barrida y la atajada viajan como contadores acumulados (`kickCount`, `slideCount`, `saveCount`) más la potencia del último pateo. Si un paquete se pierde, el cliente igual nota que el contador subió y reproduce el sonido. El primer snapshot solo sincroniza los contadores, para que al unirse a mitad de partido no suene todo junto.
- **Aplicación en el cliente:** `applySnapshot()` copia el estado sobre la simulación local. Reutiliza `setReplaySliding` y `setReplayDive` de la repetición, y suma `PlayerEntity.setDisplayKickPower()` (barra de carga del host) y `GoalkeeperEntity.setRemoteDiving()` (sonido de la estirada).

2.4 Input del cliente y robustez

- El input del cliente viaja como `INPUT` y se arma con el mismo `PlayerInput` de siempre, que ya tenía número de secuencia pensado para este paso.
- **El host no confía en el cliente:** `clampAxis` limita los ejes a [−1, 1] y convierte NaN en 0; se descartan los `INPUT` con secuencia vieja o repetida; solo se escucha al cliente aceptado.
- **Eventos pegajosos también en red:** `kickReleased`, `switchPlayer` y `slidePressed` se acumulan con OR en el host hasta que la simulación los consume, el mismo problema que la MK4 había resuelto para el teclado.
- **Cliente conectado a una sola dirección:** `channel.connect(...)` filtra los paquetes que llegan de otras direcciones.

2.5 Lobby y flujo del partido online

- **Dos filas nuevas** (el lobby pasó de 5 a 7): “ONLINE: CREAR PARTIDA” y “ONLINE: UNIRSE (IP del host)”; dificultad, duración y volumen pasaron de las filas 2–4 a las 4–6.
- **Crear partida:** ENTER abre el puerto UDP 7777 y muestra la IP local (`NetInfo.localAddress()`, primera IPv4 privada de una interfaz activa). Si el puerto está ocupado, informa que puede haber otro juego abierto.
- **Unirse:** se escribe la IP del host (dígitos, “.” y “:”, hasta 21 caracteres; formato `ip` o `ip:puerto`) y ENTER conecta. La última IP escrita se guarda entre sesiones (`Settings.lastHostIp`). Con la fila de IP elegida, los atajos 1 y 2 se desactivan para poder escribir números.

- **Estado de la conexión en pantalla** (`drawNetStatus`): “ESPERANDO AL RIVAL”, “EL RIVAL SE DESCONECTO”, “CONECTANDO...”, “NO SE PUDO CONECTAR” y “SE PERDIO LA CONEXION”. ESC vuelve al menú y cierra la conexión (también al ocultar o liberar la pantalla).
- **Sin pausa online:** P se desactiva porque pausar al host congelaría al rival. El reinicio solo existe como revancha del host.
- **Modelo de modos:** `GameMode` suma `ONLINE_HOST` y `ONLINE_CLIENT` con el método `isOnline()`, y el jugador rojo pasa a ser humano en todo modo distinto de `ONE_PLAYER`.

3. Problemas del online y cómo se corrigieron

Estos tres puntos eran problemas de la primera versión online (MK4.5) y son lo propio de la MK5.

3.1 El cliente veía todo a saltos

- **Causa:** el host manda ~30 snapshots por segundo y el cliente dibuja a 60 fps. Cada snapshot se aplicaba crudo apenas llegaba, así que el movimiento se veía entrecortado y el jitter de la red se notaba directamente.
- **Solución:** nueva clase `net/SnapshotInterpolator` (Java puro). Guarda los últimos 8 snapshots en un buffer circular con su hora de llegada y dibuja el estado de **hace 70 ms** (`DELAY_NANOS`), mezclando linealmente los dos que lo rodean.
- **Saltos grandes:** si un valor se movió más de 4 m entre dos snapshots (`TELEPORT_METERS`, por ejemplo el saque tras un gol) no se interpola y se pasa directo al valor nuevo.
- **Datos discretos:** fase, marcador, contadores y banderas se toman del snapshot más cercano en el tiempo; el ángulo de la pelota se interpola por el camino más corto.
- **Si el siguiente snapshot se retrasa:** se extrapola con la velocidad hasta un máximo de 100 ms.
- **Costo:** ~70 ms de retraso visual extra a cambio de movimiento continuo; se ajusta con `SnapshotInterpolator.DELAY_NANOS`. En `MatchScreen`, cada frame hace `push`, `sample` y `applySnapshot`; aunque `sim.step()` no simula en el cliente, rastros, sonido y confeti siguen corriendo.

3.2 Nadie sabía su ping

- **Causa:** la versión online no daba ninguna información de la conexión.
- **Solución:** protocolo v2 con `PING` / `PONG`. El cliente manda un `PING` por segundo con una marca de tiempo; el host la devuelve en un `PONG`. El ping se calcula como la diferencia con el reloj local (la resta en `int` tolera la vuelta del contador), se descartan valores fuera de 0–10 s y se suaviza (70 % valor anterior + 30 % medición nueva).
- **Ambos lo ven:** el cliente incluye su último ping medido en cada `PING`, así que el host también lo conoce (`getClientPingMs()`). Se muestra como “PING xx ms” en la esquina inferior izquierda: verde por debajo de 60 ms, amarillo por debajo de 120 ms y rojo por encima.
- **Compatibilidad:** `VERSION = 2`. Una MK4.5 y una MK5 no se conectan entre sí (el cliente viejo muestra “no se pudo conectar”); ambos jugadores deben tener la MK5.

3.3 No había forma de jugar otra

- **Causa:** al terminar el partido online había que salir al menú y reconectar.

- **Solución:** revancha. Con el partido en `FULL_TIME` y el cliente conectado, el host presiona **R** y empieza uno nuevo sin cortar la conexión (`restartMatch()`); el cliente recibe el partido nuevo por snapshots.
- **Panel final:** el host ve "R revancha - ESC menú principal" y el cliente ve "Esperando revancha del host - ESC menú principal".

4. Pantalla completa y cámara

4.1 Un jugador y online a pantalla completa

- Contra la IA ya no se divide la pantalla: se ve toda la cancha en un solo panel. **Solo el modo 2 jugadores local sigue dividido.** En online, cada jugador ve su propio panel completo (host = AZUL, cliente = ROJO).
- **IA nítida:** como no hay un segundo humano al que distinguir, la IA se dibuja sin difuminado ni anillo (`drawSharpRival`; `renderPane` recibe el parámetro `sharpRival`). El online mantiene el rival difuminado.
- **Etiqueta de esquina:** "VOS - EQUIPO AZUL (HOST)", "VOS - EQUIPO ROJO" o "VOS - EQUIPO AZUL vs IA", según el modo.

4.2 Cámara con más campo

- `FULL_VIEW_HEIGHT` = 20 m de alto visibles (antes 16 m con `SPLIT_VIEW_HEIGHT`). `updateCamera` ahora admite una altura de vista variable.
- `FULL_VIEW_BALL_BIAS` = 0,3: la cámara sigue al jugador pero se inclina un 30 % hacia la pelota para no perder la jugada (no se aplica durante la repetición).

4.3 Textos y detalles menores

- **Arreglo:** el recordatorio de controles del saque, en online, mostraba las teclas del Jugador 2 (flechas) aunque online se juega con WASD; ahora muestra las correctas.
- El título del marcador pasó de "Rivales F.C. (Local 2v2)" a "Rivales F.C." (`MATCH_TITLE`).
- La descripción de "1 JUGADOR" en el lobby aclara que es pantalla completa, y el diseño del lobby se compactó (tarjetas de 70 px y filas de 38 px) para que entren las 7 filas.

5. Ajustes de balance

La MK5 cambia dos mecánicas centrales; todo está en `Constants.java`.

5.1 Barrida nerfeada

Parámetro	MK4 → MK5	Efecto
`SLIDE_SPEED`	9,5 → 8,0 m/s	Barrida más lenta
`SLIDE_DURATION`	0,35 → 0,30 s	Alcance total de ≈ 3,3 m a ≈ 2,4 m
`SLIDE_COOLDOWN`	1,0 → 1,8 s	Ya no se puede spamear
`SLIDE_RECOVERY_TIME`	nuevo: 0,35 s	Queda en el piso sin moverse ni patear tras la barrida
`SLIDE_REACH`	0,9 → 0,8 m	Alcance a la pelota más corto (antes usaba el de pateo)

Parámetro	MK4 → MK5	Efecto
`SLIDE_KICK_POWER`	0,35 → 0,22	Toque más flojo sobre la pelota

- **Implementación:** `PlayerEntity` suma `recoveryTimer`, `isRecovering()` e `isSlideReaching()`; `handleSlideTackle` usa el alcance propio de la barrida; `resetState()` también limpia la recuperación.
- **Resultado:** una barrida errada deja vulnerable al jugador, y la barrida deja de ser un botón para spamear.

5.2 Tiro buffeado y apuntable

Parámetro	MK4 → MK5	Efecto
`SHOT_IMPULSE_MAX`	2,9 → 3,8	Potencia máxima de ≈ 38 a ≈ 50 m/s (el tope de Box2D es ≈ 60 m/s)
`KICK_CHARGE_TIME`	1,0 → 0,75 s	Se llega antes a la potencia máxima
`KICK_RANGE`	0,9 → 1,05 m	Pateo más permisivo
`AIM_ASSIST`	nuevo: 1,2	Peso de la dirección en que te movés al patear

- **Se puede apuntar:** antes el remate salía siempre en línea recta jugador → pelota. Ahora `handleKick` suma al vector jugador → pelota la dirección de movimiento al soltar el botón (escalada por `AIM_ASSIST` y por la intensidad del movimiento) y normaliza el resultado.
- **Salvaguardas:** si te movés hacia atrás de la pelota (producto escalar ≤ 0) la dirección se ignora, y los movimientos menores a 0,05 también. Tirado en el piso (barrida o recuperación) no se puede patear.
- **Efecto sobre la IA:** `SupportAI` no cambió, pero lee `KICK_RANGE`, `KICK_CHARGE_TIME` y `SLIDE_COOLDOWN`, así que hereda el rebalanceo.
- **A vigilar:** con tiros más fuertes los arqueros IA atajan menos; si queda desparejo, se ajustan `GK_SPEED` y `GK_DIVE_SPEED`.

6. Verificación

- **NetSelfTest** (nuevo, en `core/src/test`): 26 pruebas por loopback, sin LibGDX. Cubren la conexión, el input (ejes, sprint, pateo “pegajoso” que se consume una sola vez), el snapshot idéntico en todos los campos, el descarte de snapshots viejos, 1.200 snapshots seguidos con más de 95 % de llegada, el ping medido y su conocimiento por parte del host, el interpolador (vacío, mitad de camino, dato discreto, antes del primero, extrapolación acotada, teletransporte y buffer circular), la basura ignorada, el segundo cliente rechazado, el BYE y el timeout sin host. Resultado informado por el equipo: **26/26**.
- **Cómo correrlas sin LibGDX:** desde `core/src`, compilar `net/*.java`, `PlayerInput.java` y `NetSelfTest.java` con `javac -encoding UTF-8` y ejecutar `com.rivalesfc.game.net.NetSelfTest`.
- **Pendiente de probar:** los cambios en `MatchScreen` (interpolador, ping, tecla R de revancha, texto del panel final, pantalla completa en 1 jugador y cámara nueva) y la integración en

`MatchSimulation` y `LobbyScreen` se escribieron en un entorno sin LibGDX y no se compilaron ahí. Hay que probarlos en dos ventanas o dos máquinas antes de darlos por buenos.

7. Mapa de cambios por archivo

Archivo	Líneas	Cambio principal
Nuevos		
`net/Protocol.java`	0 → 168	Formato de los mensajes UDP (protocolo v2)
`net/Snapshot.java`	0 → 67	Estado del partido que viaja del host al cliente
`net/SnapshotInterpolator.java`	0 → 170	Interpolación y extrapolación en el cliente
`net/NetHost.java`	0 → 198	Lado host: un cliente, input, snapshots, ping
`net/NetClient.java`	0 → 181	Lado cliente: conexión, input, snapshots, ping
`net/NetInfo.java`	0 → 29	IP local para mostrarle al host
`test/.../NetSelfTest.java`	0 → 177	26 pruebas de red por loopback
Modificados		
`screens/MatchScreen.java`	1.265 → 1.481	Red, ping, revancha, pantalla completa, cámara, rival nítido
`sim/MatchSimulation.java`	605 → 742	Snapshots, modo cliente, pateo apuntable, alcance de barrida
`screens/LobbyScreen.java`	268 → 356	Filas online, escritura de IP, estado de conexión
`entities/PlayerEntity.java`	195 → 221	Recuperación tras la barrida, alcance de barrida, carga remota
`Constants.java`	138 → 144	Balance de barrida y tiro, cámara de pantalla completa, título
`GameMode.java`	17 → 25	ONLINE_HOST, ONLINE_CLIENT e isOnline()
`Settings.java`	104 → 108	Última IP de host guardada
`entities/GoalkeeperEntity.java`	218 → 223	setRemoteDiving() para el cliente

Sin cambios: `RivalesFCGame`, `SupportAI`, `AudioFactory`, `Ball`, `Field`, `PlayerInput`, `MatchStats`, `ReplayFrame`, `MotionTrail`, `PixelArtFactory`, `BlurShader`, `DesktopLauncher` y los `build.gradle`. También se actualizaron el `README.md` y se agregó `INFORME_MK5.md` junto al `INFORME_MK4.md`.

> **Continuidad de diseño**
> Varias soluciones de la MK5 reutilizan decisiones anteriores: el host corre el mismo núcleo de simulación a **tick fijo de 30 Hz** pensado desde la Etapa 1, el input del cliente viaja en el mismo `PlayerInput` (con número de secuencia desde el principio), los eventos de red son **"pegajosos"** como en la MK4, y la aplicación de snapshots reutiliza `setReplaySliding` y `setReplayDive` de la repetición. Además, el cliente no tiene física propia: solo recibe y dibuja datos de presentación.

Resumen de evolución

Versión	Objetivo	Principales incorporaciones
MK1	Prototipo	Física, movimiento, disparos, arquero y split-screen
MK2	Jugabilidad y estabilidad	Rediseño gráfico, corrección de física, barrida, lobby e IA
MK3	Experiencia y feedback	Audio, screen shake, animaciones, posesión y tutorial
MK4	Cierre de pendientes y pulido	Correcciones de entrada y potencia, estirada, IA con decisiones, repetición, estadísticas y MVP, shader de blur, opciones
MK5	Online 1v1 y rebalanceo	Online 1v1 por UDP (host + cliente), interpolación, ping y revancha, protocolo v2, 1 jugador a pantalla completa, barrida nerfeada y tiro buffeado

Estado actual

Con la MK4, Rivales FC cerró todos los pendientes que la MK3 había dejado fuera de alcance y corrigió los errores que alteraban la sensación de juego: el núcleo local quedó estable y completo. Con la MK5, el juego da el primer paso a la red: dos jugadores pueden enfrentarse en un **partido online 1v1** (un host autoritativo y un cliente), con movimiento interpolado, ping visible y revancha, además de una barrida y un tiro rebalanceados.

A lo largo del proyecto, la evolución fue progresiva: la MK1 estableció la base física y de control, la MK2 resolvió los problemas técnicos y agregó mecánicas, la MK3 mejoró la respuesta audiovisual y la información disponible, la MK4 diagnosticó y corrigió por qué el juego no se sentía del todo bien, y la MK5 llevó ese núcleo a la red y lo pulió para que el online se sienta continuo.

Fuera de alcance de esta etapa

- **Predicción del lado del cliente:** con ping alto, el jugador propio responde con retraso (ping + ~70 ms).
- Lobby en red, formaciones y chat (Etapa 3).
- Faltas, tarjetas, offside y tiro con efecto completo (Etapa 4).
- Arte y sonido definitivos (Etapa 5).
- Pases a compañeros reales, que requieren más de un jugador de campo por equipo.

Apéndice: controles de la MK5

Acción	Jugador 1 (AZUL)	Jugador 2 (ROJO)
Mover	W A S D	Flechas
Sprint	Shift izquierdo	Ctrl derecho
Cargar y patear	Mantener y soltar Espacio	Mantener y soltar Enter
Barrida	Ctrl izquierdo	Shift derecho

Acción general	Tecla
Pausar / reanudar	P (no disponible online)
Reiniciar el partido	R (online: solo el host, como revancha al terminar el partido)
Saltear la repetición de gol	TAB
Silenciar / activar sonido	M
Menú principal	**ESC**
Ver cuerpos de Box2D (debug)	F1

Online (nuevo en la MK5)	Tecla / acción
Mover, sprint, cargar y patear, barrida	Host (AZUL) y cliente (ROJO) usan el esquema del Jugador 1: W A S D · Shift izquierdo · Espacio · Ctrl izquierdo
Crear partida (host)	En el lobby, fila “ONLINE: CREAR PARTIDA” ⟶ ENTER (muestra la IP local)
Unirse (cliente)	Fila “ONLINE: UNIRSE”: escribir la IP del host (dígitos, “.” y “:”), BACKSPACE borra, ENTER conecta
Revancha	R (solo el host, al terminar el partido)
Menú principal	ESC (cierra la conexión)